

Lesson 9.1 — Demo Runbook (презентація + жива практика)

Цей файл — **єдина інструкція** під час міта: не перемикайся на `lesson_9_1_practice.md`; усі кроки нижче самодостатні.

Матеріали лише для підготовки (не під час демо): конспект `lesson_9_1.md`, слайди `lesson_9_1_presentation.pdf`.

0) Підготовка середовища (до заняття або за 5 хв до демо)

1. Переконайся, що контейнери запущені:
 - `docker compose ps`
 - якщо потрібно: `docker compose up -d`
 - якщо порт `3306` зайнятий іншим MySQL — або зупини конфліктний контейнер, або тимчасово зміни мапінг порту в `docker-compose.yml` (це локальна проблема машини, не коду).
2. Перевір маршрути API:
 - `docker compose exec -T php php artisan route:list --path=api/v1`
3. Очікувані ключові URL для уроку:
 - `GET/POST /api/v1/accounts`, `GET/DELETE /api/v1/accounts/{account}`
 - `GET /api/v1/accounts/{account}/payments` — список платежів рахунку (приклад із уроку 9.1)
 - `GET/POST /api/v1/payments`, `GET/DELETE /api/v1/payments/{payment}`
4. Тести (SQLite in-memory з `phpunit.xml`) — у контейнері або локально з Composer:
 - `docker compose exec -T php php artisan test tests/Feature/Api/V1`
 - або: `php artisan test tests/Feature/Api/V1`

Фраза для аудиторії: «Урок 9.1 — про системне мислення щодо швидкості: спочатку вимірюємо й знаходимо вузьке місце, потім один маленький крок оптимізації.»

1) Базовий ритм «слайд → практика → назад до слайдів»

Після кожного слайду з міткою «практика»:

1. 30–60 сек — теза на слайді.
 2. Виконай блок команд/питань з цього runbook.
 3. 1–2 речення підсумку.
 4. Повернись до наступного слайду.
-

2) Блок А — аналіз API-ендпоінта (у практиці: **Слайд 8а** / Блок 3)

Мета: показати шлях запиту: HTTP → контролер → сервіс → репозиторій → SQL → JSON.

Що сказати вголос (без коду на слайді):

- У фокусі приклад з конспекту: **список платежів рахунку** — **GET** `/api/v1/accounts/{id}/payments`.
- Питання для аналізу: скільки SQL-запитів, чи є пагінація, чи не тягнемо зайві поля, чи можливий N+1, чи є кеш.

Жива демонстрація (тільки read, без зміни коду):

1. Створи рахунок (якщо треба «чисті» дані):
 - `curl -s -X POST http://localhost/api/v1/accounts -H "Content-Type: application/json" -d '{"balance":"5000.00"}'`
2. Запит списку платежів рахунку (підстав ID з відповіді):
 - `curl -s "http://localhost/api/v1/accounts/ID/payments?per_page=20"`
3. Покажи в JSON: **data**, **meta**, **links** — **пагінація вже в SQL**, не «обрізка масиву в PHP».

Де це в коді (для твого коментаря, опційно):

- Маршрут: `routes/api.php` — `accounts/{account}/payments`
- `AccountController::payments` → `AccountPaymentsIndexRequest` + `AccountPaymentsQueryDTO` → `PaymentService::listPaymentsForAccount` → `PaymentRepository::paginateByAccountId` (`eager with( 'account' )`) — тема N+1 в наступному блоці)

Перехід: «Описали потік; далі — що кешувати, а що ні.»

3) Блок В — кеш vs не-кеш (у практиці: **Слайд 11а** / Блок 4)

Мета: відокремити довідники/агрегати від real-time грошей.

Що сказати (коротко):

- **Можна** (зазвичай): довідники (статуси, типи), рідкі зміни, агреговані дашборди з TTL, read-only списки.
- **Обережно / зазвичай ні:** поточний баланс, щойно створений платіж у критичному сценарії, персональні дані з жорсткою консистентністю.

Практика без сторонніх сервісів (тільки обговорення):

- Попроси групу: «Що з **GET** `/accounts/{id}/payments` кешували б ви на 30 с, а що ні?»

Перехід: «Кеш — компроміс швидкість/актуальність; N+1 б'є швидше за вгадування з кешем.»

4) Блок С — N+1 (у практиці: **Слайд 14а** / Блок 5)

Мета: пояснити 1 + N запитів і лінійне повільнання.

Що сказати:

- Псевдокод із практики: цикл по платежах і ліниве завантаження `account` → десятки/сотні запитів.

Зв'язок із нашим проектом:

- У репозиторії для списку платежів рахунку використано `with('account')` перед `paginate`, щоб зв'язок не підвантажувався в циклі серіалізації (на навчальному прикладі).

Перехід: «Один вимірюваний крок краще за десять "оптимізацій на око".»

5) Блок D — перший крок оптимізації (у практиці: **Слайд 17a** / Блок 6)

Мета: вибрати **один** крок: eager loading, пагінація, індекс, менший JSON.

Що запропонувати на прикладі списку платежів:

1. Уже є: **пагінація** (`per_page` до 100 у валідації).
2. Уже є: **eager loading** рахунку для колекції платежів.
3. Наступний вимірюваний крок (на майбутні уроки 9.2–9.3): перевірити **індекси** по `payments.account_id` та (`account_id, created_at`) через план запиту / профайлер.

Як аргументувати вимірювання:

- До/після: час відповіді, кількість SQL, навантаження на БД.

Перехід: «Підсумок модуля 9 — оптимізація після метрик, не до них.»

6) Перевірка логів (без сторонніх систем — тільки Docker)

Після будь-якого **POST** (`accounts/payments`) можна показати структуровані логи:

- `docker compose exec -T php sh -lc 'tail -n 80 storage/logs/laravel.log'`

Що підкреслити: кореляція з модулем 8 (`correlation_id, endpoint`) — швидкість і помилки шукаємо в логах/метриках, а не на здогад.

7) Фінальний чек-лист перед завершенням уроку

- Пояснено: продуктивність = швидкість + стабільність + передбачуваність.
 - Розібрано вузькі місця: БД, N+1, кеш, об'єм відповіді.
 - Показано живий **GET** списку платежів по рахунку.
 - Запущено тести API на **SQLite** (`tests/Feature/Api/V1`).
 - Показано читання логів через контейнер.
-

8) Швидкий повтор демо (якщо час піджимає)

1. `docker compose ps`
2. `docker compose exec -T php php artisan route:list --path=api/v1/accounts`

3. `curl -s "http://localhost/api/v1/accounts/1/payments?per_page=10"`
 4. `docker compose exec -T php php artisan test tests/Feature/Api/V1`
-

Примітка про PDF-презентацію

Файл `lesson_9_1_presentation.pdf` під час демо лише **листаєш**; усі «переходи на практику» — за цим `runbook`, без другого текстового файлу поруч.